Visualization Literacy Analysis

Derek Harter, Shulan Lu

11/12/2025

Read in data files

Raw data is in the `../AccData` subdirectory in two excel spreadsheet files per participant. The following code combines into single dataframe of all data.

```
#grab files from google drive (only have to do this once)
# source("getFromGoogleDrive.R")

#get the first 6? characters of each data file
#get unique values of these
# this is list of subject ids

raw_file_names <- list.files("../AccData")
first_six <- substr(raw_file_names, 1, 6)
sub_ids <- unique(first_six)


# fast_RTs <- data.frame(ParticipantId = character(),
#                        TrialName = character(),
#                        type = character(),
#                        time = numeric()
# )

all_data <- NULL

for (i in 1:length(sub_ids)){

  if (grepl("~$", sub_ids[i], fixed = T)){
    next
  }

  temp_file1 <- read_xlsx(paste0("../AccData/", sub_ids[i], "_1.xlsx")) %>%
    slice(1:17) %>%
    select(-starts_with("Order")) %>%
    rename(correct = 8)

  temp_file2 <- read_xlsx(paste0("../AccData/", sub_ids[i], "_2.xlsx")) %>%
    slice(1:17) %>%
     select(-starts_with("Order")) %>%
    rename(correct = 8)

  new_temp <- temp_file1 %>%
    bind_cols(temp_file2$correct) %>%
```

```
    rename(Correct_1 = correct, Correct_2 = "...9") %>%
    mutate(AnswerRT = TimeToBeginInput - TimeToReadQuestion)


  all_data <- all_data %>%
    bind_rows(new_temp)

}

all_data <- all_data %>%
  rename(readRT = TimeToReadQuestion, totalRT = TimeToBeginInput)

#read in trialtype key (I created this from an early version of the previous paper)
trial_type_key <- read.csv("../trial_type_key.csv", stringsAsFactors = F)

all_data <- all_data %>%
  mutate(TrialType = trial_type_key$TrialType[match(TrialName, trial_type_key$TrialName)]) %>%
  mutate(TrialType = paste0("Type",TrialType))


# remove temporary dataframes to keep workspace cleaner as we go forward
rm(new_temp)
rm(temp_file1)
rm(temp_file2)
rm(trial_type_key)
```

Basic checks

Just a sanity check of all the data. Summary of number of subjects in each of the 3 condition types.

```
#how many participants per condition
all_data %>%
  group_by(ParticipantId, Condition) %>%
  summarize(ntrials = n()) %>%
  group_by(Condition) %>%
  summarize(nsubs = n()) %>%
  kable()
```

```
## `summarise()` has grouped output by 'ParticipantId'. You can override using the `.groups`
## argument.
```

Condition	nsubs
VR	50
VR Monitor	39
VR Monitor Stereo	33

```
#why is the balance so off?
```

Remove outliers based on RT

Removing outliers, resulting in data frame named `all_data_no_outliers`.

1. All trials where the `AnswerRT` is less than 2 seconds are removed first, which assumes these reactions were too fast so were done without an actual considered response.

2. All trials where the `AnswerRT` is 3 standard deviations different from the mean are removed next.

Table shows summary of mean and standard deviation of answer RT by each of the trial question types, as well as the 3 standard deviation upper and lower bound of the reaction time.

Displayed histogram is a sanity check of the number of trials each participant in all of the data have done. Most participants should do 16 trials. Less than that are from dropped data for outliers.

```
#removing on trial-by-trial basis


#remove answerRTs below 2000ms first
all_data_remove <- all_data %>%
  filter(AnswerRT >= 2000)
dim(all_data)[1]
```

```
## [1] 2074
```

```
dim(all_data_remove)[1]
```

```
## [1] 2067
```

```
#drops 7 trials

# also remove AnswerRT that are more than 3 standard deviations away from
# the mean
all_data_no_outliers <- all_data_remove %>%
  group_by(TrialName) %>%
  filter((!(abs(AnswerRT - mean(AnswerRT)) > 3.0*sd(AnswerRT))))
dim(all_data_no_outliers)[1]
```

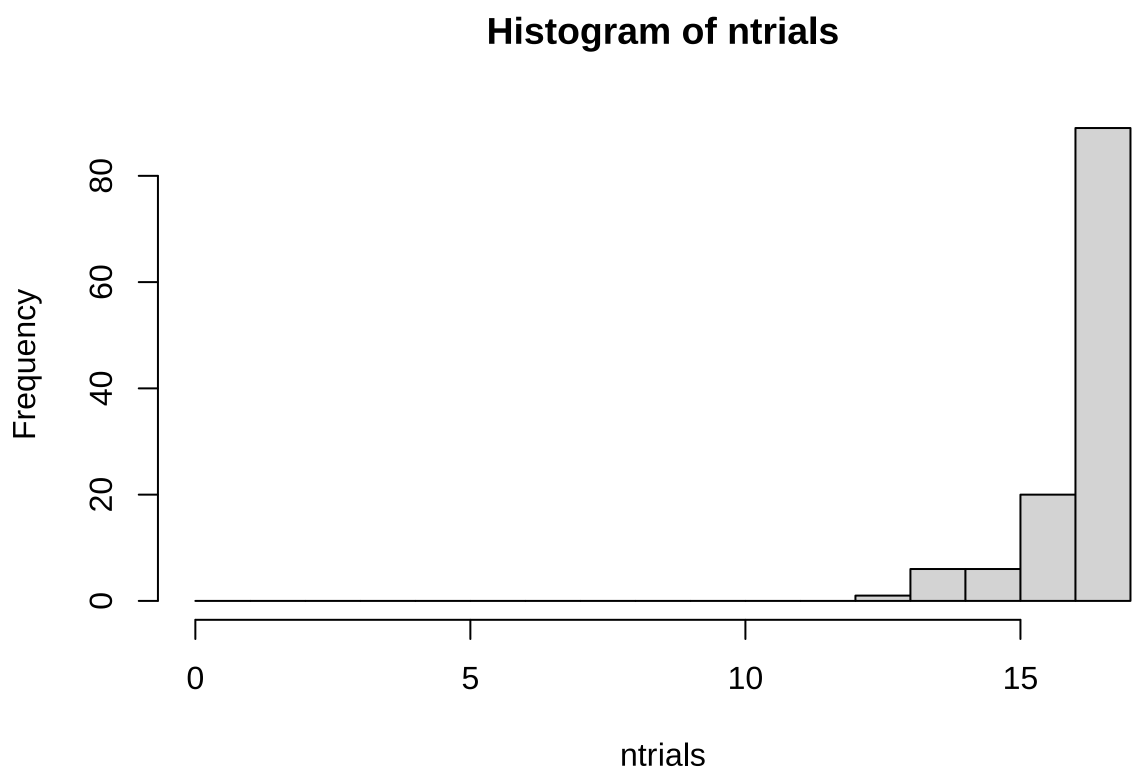
```
## [1] 2020
```

```
#drops 47 more trials


rt_data_summary <- all_data_no_outliers %>%
  group_by(TrialName) %>%
  summarize(meanAnswerRT = mean(AnswerRT, na.rm = T),
            sdAnswerRT = sd(AnswerRT, na.rm = T),
            LB = meanAnswerRT - 3*sdAnswerRT,
            UB = meanAnswerRT + 3*sdAnswerRT)
kable(rt_data_summary)
```

TrialName	meanAnswerRT	sdAnswerRT	LB	UB
BarChartQ1	26701.293	13004.715	-12312.853	65715.44
BarChartQ2	15688.149	10925.288	-17087.717	48464.01
BarChartQ3	14701.729	8225.461	-9974.653	39378.11
BarChartQ4	8962.825	5155.541	-6503.798	24429.45
LineChartQ1	46326.506	23587.872	-24437.109	117090.12
LineChartQ2	38246.295	23995.233	-33739.405	110232.00
LineChartQ3	25274.621	14260.815	-17507.824	68057.07
LineChartQ4	13372.279	13262.905	-26416.437	53161.00
LineChartQ5	27265.489	15938.188	-20549.075	75080.05
ScatterplotQ1	29913.045	18641.608	-26011.778	85837.87
ScatterplotQ2	27968.794	20282.704	-32879.316	88816.90
ScatterplotQ3	42887.765	26920.371	-37873.347	123648.88

TrialName	meanAnswerRT	sdAnswerRT	LB	UB
ScatterplotQ4	33751.255	19808.692	-25674.820	93177.33
ScatterplotQ5	58482.379	38675.866	-57545.219	174509.98
SurfacePlotQ1	53283.033	29477.074	-35148.188	141714.25
SurfacePlotQ2	61771.846	43154.455	-67691.520	191235.21
SurfacePlotQ3	44518.559	29753.055	-44740.607	133777.72

```
rm(rt_data_summary)


all_data_no_outliers %>%
  group_by(ParticipantId) %>%
  summarize(ntrials = n()) %>%
  with(hist(ntrials, breaks = 0:17))
```




```
rm(all_data_remove)
##maybe consider replacing outliers with means instead of removing them?
```

Extract Only One Type of Chart

Create a dataframe from the data with no outliers that contains only XChartQ1 through XChartQ4 trials for the following analysis.

We replace the `all_data_no_outliers` dataframe with one containing only the selected TrialName question type here, so past this point we are analyzing only this particular TrialName type.

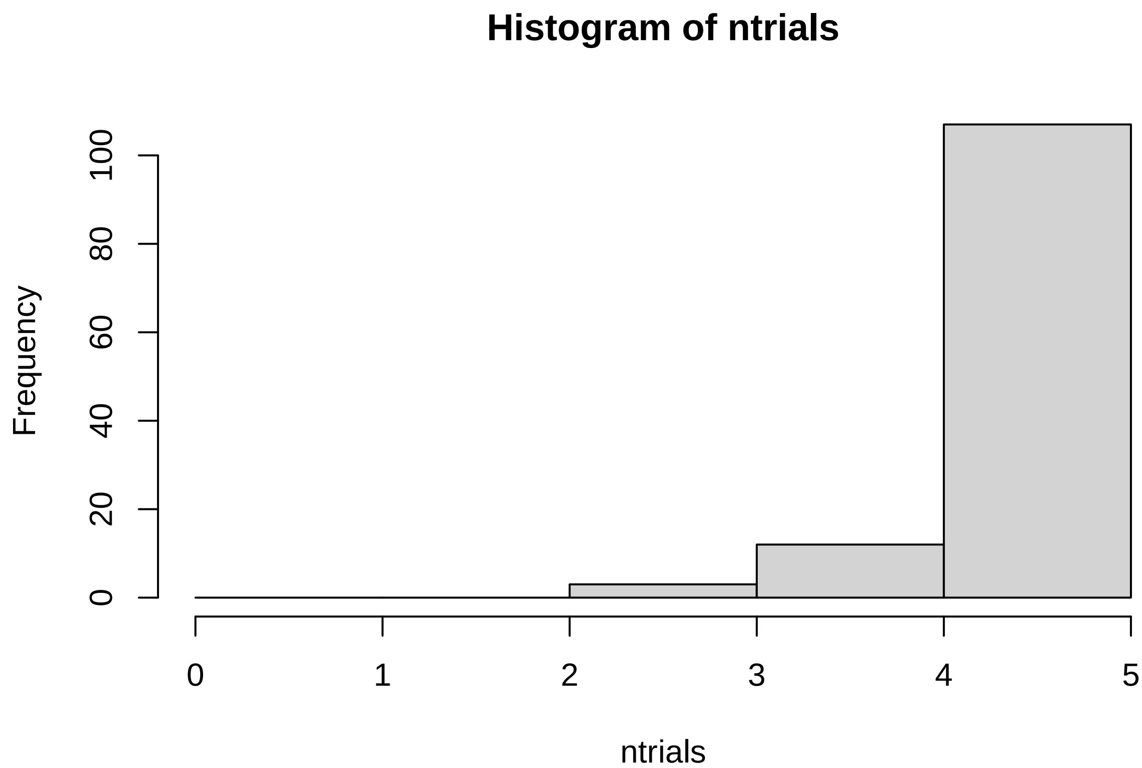
We show the table again of the mean RT for this question type. The table should match the one done with all of the data, but we should only have the particular question trial name type now in the data.

We also again plot the histogram of the number of trials. Participants should have seen 4 of each particular question type, but after removing outliers some only have 3, 2 or 1 remaining.

```
all_data_no_outliers <- all_data_no_outliers %>%
  filter(grepl("ScatterplotQ[0-9]", TrialName))

rt_data_summary <- all_data_no_outliers %>%
  group_by(TrialName) %>%
  summarize(meanAnswerRT = mean(AnswerRT, na.rm = T),
            sdAnswerRT = sd(AnswerRT, na.rm = T),
            LB = meanAnswerRT - 3*sdAnswerRT,
            UB = meanAnswerRT + 3*sdAnswerRT)
kable(rt_data_summary)
```

TrialName	meanAnswerRT	sdAnswerRT	LB	UB
ScatterplotQ1	29913.05	18641.61	-26011.78	85837.87
ScatterplotQ2	27968.79	20282.70	-32879.32	88816.90
ScatterplotQ3	42887.77	26920.37	-37873.35	123648.88
ScatterplotQ4	33751.25	19808.69	-25674.82	93177.33
ScatterplotQ5	58482.38	38675.87	-57545.22	174509.98

```
rm(rt_data_summary)


all_data_no_outliers %>%
  group_by(ParticipantId) %>%
  summarize(ntrials = n()) %>%
  with(hist(ntrials, breaks = 0:5))
```



Compare Read RT

Compare the reading reaction times for the selected question type.

Extract wide dataframe

In this cell we convert the data to a wide format, selection only Condition, TrialType and the mean_readRT, and widening the TrialType column.

We have also filled in na/missing values here as a final step here. Maybe we should instead just change from removing outliers to replacing with mean. Also the filling missing values code here should be easier to do, needs to be fixed. But result should be that any na in the resulting wide table is replaced with the mean for the particular Type.

```
#compare read times (should be no differences of condition)

trial_type_means <- all_data_no_outliers %>%
  group_by(ParticipantId, Condition, TrialType) %>%
  summarize(mean_readRT = mean(readRT), n = n())
```

```
## `summarise()` has grouped output by 'ParticipantId', 'Condition'. You can override using
## the `.groups` argument.
```

```
# first, make .csv files in wide format to double check in statview
read_rt <- trial_type_means %>%
  select(ParticipantId, Condition, TrialType, mean_readRT) %>%
  pivot_wider(names_from = TrialType,values_from = mean_readRT)
```

```r
# fill in na values with mean for type1, type2 and type3 RT
# the following is supposed to work, but we get NaN from calculation of mean here, not sure why...
#read_rt_type_wider_clean <- read_rt_type_wider %>% mutate(across(Type1, ~replace_na(., mean(., na.rm=T
read_rt <- read_rt %>% replace_na(list(Type1 = mean(read_rt$Type1, na.rm=TRUE)))
read_rt <- read_rt %>% replace_na(list(Type2 = mean(read_rt$Type2, na.rm=TRUE)))
read_rt <- read_rt %>% replace_na(list(Type3 = mean(read_rt$Type3, na.rm=TRUE)))


read_rt_summary <- read_rt %>%
  group_by(Condition) %>%
  summarize(meanType1 = mean(Type1, na.rm = T),
            meanType2 = mean(Type2, na.rm = T),
            meanType3 = mean(Type3, na.rm=T))
kable(read_rt_summary)
```

Condition	meanType1	meanType2	meanType3
VR	9358.704	9098.998	8844.438
VR Monitor	11107.944	10163.340	8918.846
VR Monitor Stereo	11456.498	10180.186	9731.548

```r
rm(read_rt_summary)

#rm(trial_type_means)
```

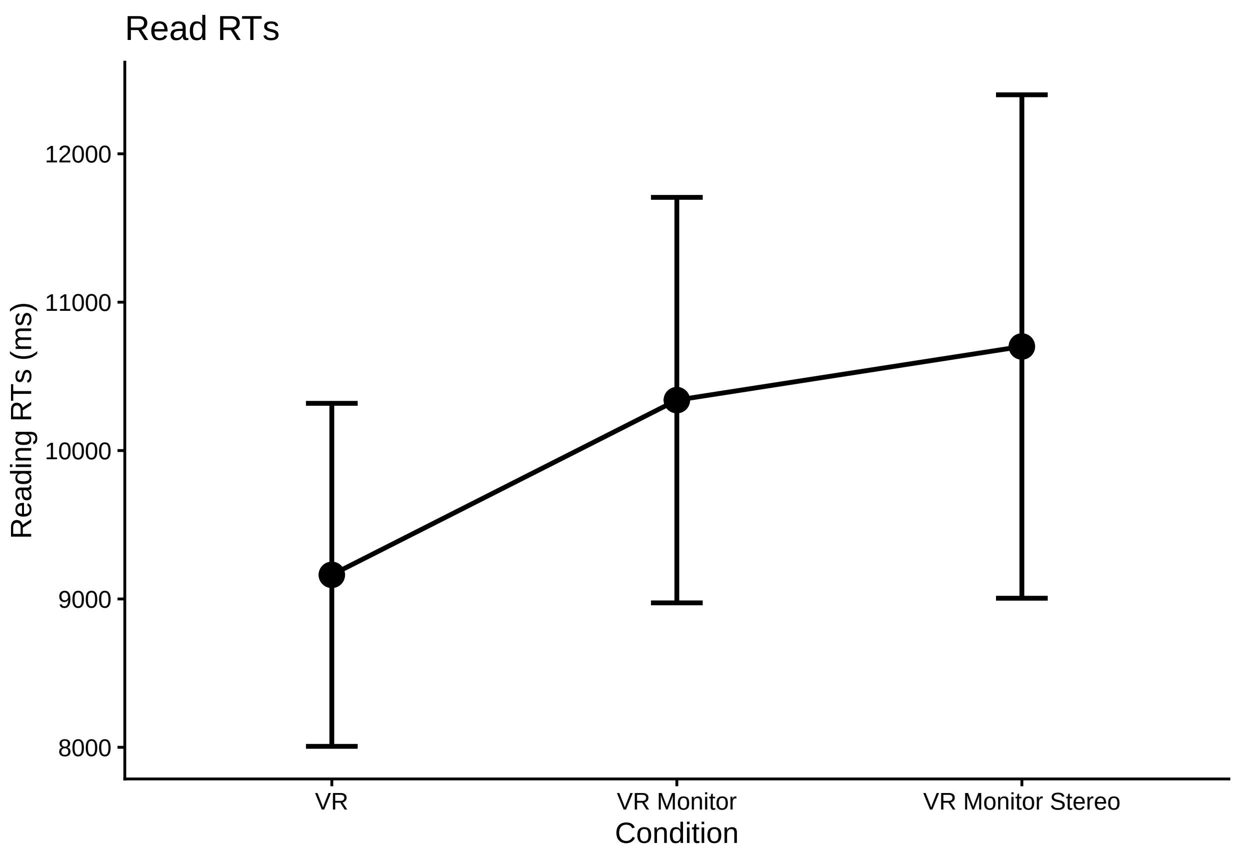
Basic plots of main effect and wider format

Plot of read RT condition main effect, as well as wider format plot by the trial types.

```r
#just condition main effect
readplot1 <- all_data_no_outliers %>%
   group_by(ParticipantId, Condition) %>%
   summarize(overallmean = mean(readRT)) %>%
   group_by(Condition) %>%
   summarize(overall_condition_mean = mean(overallmean),
             se = std.error(overallmean),
             n = n(),
             CI = qt(0.975,df=n-1)*se)
```

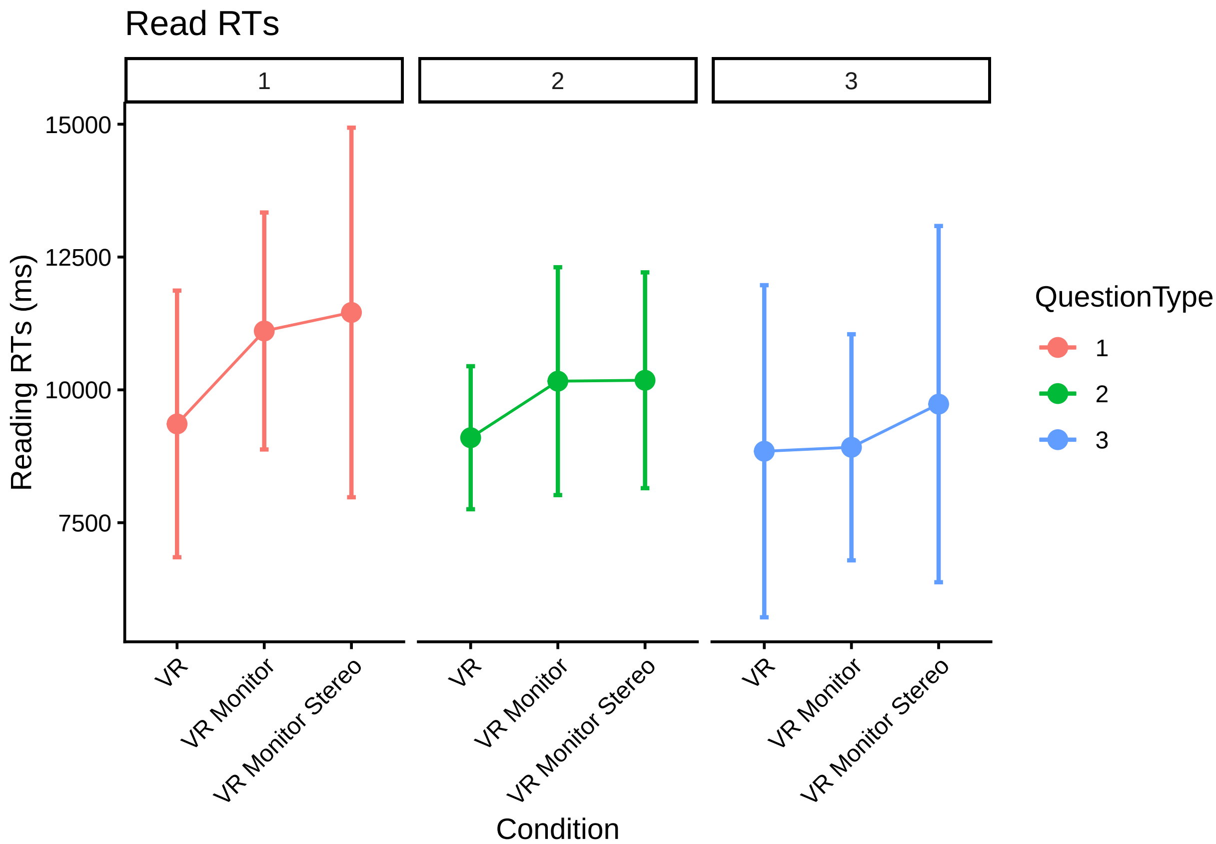
```
## `summarise()` has grouped output by 'ParticipantId'. You can override using the `.groups`
## argument.
```

```r
 ggplot(readplot1, aes(Condition,
                       overall_condition_mean,
                       group = 1,
                       ymin = overall_condition_mean - CI,
                       ymax = overall_condition_mean + CI)) +
   theme_classic() +
   geom_point(size = 4) +
   geom_errorbar(width = .15, linewidth = 0.85) +
   geom_line(linewidth = 0.85) +
   labs(y = "Reading RTs (ms)", title = "Read RTs")
```




```
rm(readplot1)

#make a little plot using wide format
superbPlot(read_rt,
    BSFactors    = "Condition",
    WSFactors    = "QuestionType(3)",
    variables    = c("Type1", "Type2", "Type3"),
    statistic    = "meanNArm",
    errorbar     = "CI",
    gamma        = 0.95,
    adjustments = list(
        purpose        = "difference"
    ),
    plotLayout = "line",
    factorOrder = c("Condition", "QuestionType")
) +
theme_classic() +
  theme(axis.text.x = element_text(angle = 45, vjust = 1, hjust=1))+
facet_wrap(vars(QuestionType))+
labs(y = "Reading RTs (ms)", title = "Read RTs")
```

```
## superb::ADVICE: Some of the groups' variances are heterogeneous. Consider using purpose="tryon".
```

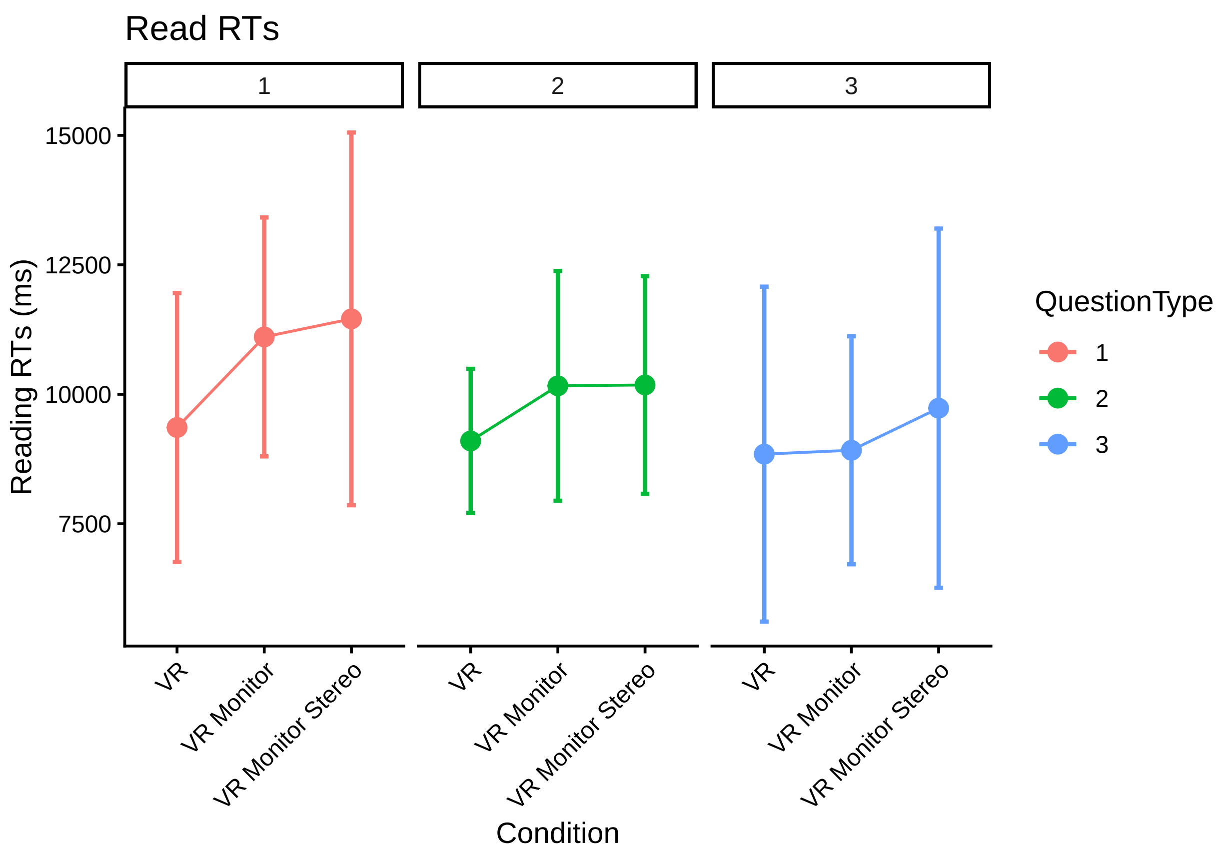


Correct wider plot heterogeneous variance

The message from the superbPlot may indicate that some of the groups' variances are heterogeneous. We can try using the `tryon` purpose option as suggested, read Unequal variances, Welch test, Tryon adjustment, and superb.

```
# make the wide plot, use tryon for purpose as suggested because of group's heterogeneous variances
superbPlot(read_rt,
    BSFactors   = "Condition",
    WSFactors   = "QuestionType(3)",
    variables   = c("Type1", "Type2", "Type3"),
    statistic   = "meanNArm",
    errorbar    = "CI",
    gamma       = 0.95,
    adjustments = list(
        purpose       = "tryon"
    ),
    plotLayout = "line",
    factorOrder = c("Condition", "QuestionType")
) +
theme_classic() +
  theme(axis.text.x = element_text(angle = 45, vjust = 1, hjust=1))+
facet_wrap(vars(QuestionType))+
labs(y = "Reading RTs (ms)", title = "Read RTs")
```

```
## superb::FYI: The tryon adjustments per measures are  Measure 1: 1.4632, Measure 2: 1.4632, Measure 3
```



Perform anova and post-hoc tests

Finally perform anova on the read RT data and various posthoc tests.

```
# need back into long format for following, but we will use out dataframe
# will filled in na/missing and make wide again here
read_rt_long <- read_rt %>%
     pivot_longer(Type1:Type3, names_to="TrialType", values_to = "mean_readRT")

read_rt_anova <- aov_ez(id = "ParticipantId",
                        dv = "mean_readRT",
                        data = read_rt_long,
                        within = "TrialType",
                        between = "Condition",
                        anova_table = list(es = "pes") #might want to double-check these
)
```

```
## Converting to factor: Condition
```

```
## Contrasts set to contr.sum for the following variables: Condition
```

```
kable(nice(read_rt_anova))
```

Effect	df	MSE	F	pes	p.value
Condition	2, 119	56889167.71	1.09	.018	.340
TrialType	1.59, 188.79	24878890.94	3.29 +	.027	.051
Condition:TrialType	3.17, 188.79	24878890.94	0.47	.008	.711

```r
#posthoc test for trial type
pairs(emmeans(read_rt_anova, "TrialType"), adjust = "Tukey")
```

```
##  contrast      estimate  SE  df t.ratio p.value
##  Type1 - Type2      827 456 119   1.815  0.1691
##  Type1 - Type3     1476 704 119   2.098  0.0946
##  Type2 - Type3      649 544 119   1.192  0.4600
##
## Results are averaged over the levels of: Condition
## P value adjustment: tukey method for comparing a family of 3 estimates
```

```r
#posthoc test for condition, still using Tukey
pairs(emmeans(read_rt_anova, "Condition"), adjust = "Tukey")
```

```
##  contrast                        estimate   SE  df t.ratio p.value
##  VR - VR Monitor                     -963  930 119  -1.035  0.5565
##  VR - VR Monitor Stereo             -1355  977 119  -1.388  0.3506
##  VR Monitor - VR Monitor Stereo     -393 1030 119  -0.381  0.9231
##
## Results are averaged over the levels of: TrialType
## P value adjustment: tukey method for comparing a family of 3 estimates
```

```r
#posthoc test for condition, others seem to be scheffe, sidak, bonferroni, dunnettx, mvt, none
pairs(emmeans(read_rt_anova, "Condition"), adjust = "scheffe")
```

```
##  contrast                        estimate   SE  df t.ratio p.value
##  VR - VR Monitor                     -963  930 119  -1.035  0.5869
##  VR - VR Monitor Stereo             -1355  977 119  -1.388  0.3847
##  VR Monitor - VR Monitor Stereo     -393 1030 119  -0.381  0.9299
##
## Results are averaged over the levels of: TrialType
## P value adjustment: scheffe method with rank 2
```

```r
rm(read_rt_long)
rm(read_rt_anova)
```

Alternative Analysis using glmm TMB package

The following analysis is for the heterogenous group condition analysis. See .

The following determines if a more complex model is justified.

```r
model_glmm_hom = glmmTMB(mean_readRT ~ Condition * TrialType + (1 | ParticipantId),
                         data=trial_type_means, family = Gamma(link="log"),
                         dispformula = ~ 1, REML = FALSE)
model_glmm_hetero = glmmTMB(mean_readRT ~ Condition * TrialType + (1 | ParticipantId),
                         data=trial_type_means, family = Gamma(link="log"),
                         dispformula = ~ Condition, REML = FALSE)


anova(model_glmm_hom, model_glmm_hetero)
```

```
## Data: trial_type_means
## Models:
## model_glmm_hom: mean_readRT ~ Condition * TrialType + (1 | ParticipantId), zi=~0, disp=~1
## model_glmm_hetero: mean_readRT ~ Condition * TrialType + (1 | ParticipantId), zi=~0, disp=~Condition
##                   Df    AIC    BIC  logLik deviance  Chisq Chi Df Pr(>Chisq)
```

```
## model_glmm_hom    11 7034.5 7077.3 -3506.3   7012.5
## model_glmm_hetero 13 7037.0 7087.6 -3505.5   7011.0 1.5349      2     0.4642
```

Try overall effect reporting

```r
fixed_effects_anova <- Anova(model_glmm_hom, type="III")
kable(fixed_effects_anova)
```

	Chisq	Df	Pr(>Chisq)
(Intercept)	13827.160744	1	0.0000000
Condition	3.227706	2	0.1991190
TrialType	2.780373	2	0.2490289
Condition:TrialType	2.897359	4	0.5751465

EMM on Condition and Interaction

If the glmm analysis is not significant, just use the homogeneous model. Otherwise report on the more complex heterogeneous model?

```r
emm_main_condition <- emmeans(model_glmm_hom,
                          specs = ~ Condition,
                          type = "response")
```

```
## NOTE: Results may be misleading due to involvement in interactions
```

```r
condition_results <- pairs(emm_main_condition)


kable(condition_results)
```

contrast	ratio	SE	df	null	z.ratio	p.value
VR / VR Monitor	0.9361376	0.0941160	Inf	1	-0.6564070	0.7887262
VR / VR Monitor Stereo	0.8691863	0.0917978	Inf	1	-1.3274606	0.3798468
VR Monitor / VR Monitor Stereo	0.9284814	0.1034362	Inf	1	-0.6660907	0.7831870

```r
emm_interaction <- emmeans(model_glmm_hom,
                          specs = ~ Condition * TrialType,
                          type = "response")

simple_effects_results <- pairs(emm_interaction, by = "TrialType")


kable(simple_effects_results)
```

contrast	TrialType	ratio	SE	df	null	z.ratio	p.value
VR / VR Monitor	Type1	0.8489659	0.0988668	Inf	1	-1.4059981	0.3377519
VR / VR Monitor Stereo	Type1	0.8218173	0.1005598	Inf	1	-1.6037336	0.2440116
VR Monitor / VR Monitor Stereo	Type1	0.9680216	0.1248027	Inf	1	-0.2520900	0.9655746
VR / VR Monitor	Type2	0.9673426	0.1126356	Inf	1	-0.2851520	0.9561679
VR / VR Monitor Stereo	Type2	0.8980415	0.1097225	Inf	1	-0.8801701	0.6528410

contrast	TrialType	ratio	SE	df	null	z.ratio	p.value
VR Monitor / VR Monitor Stereo	Type2	0.9283594	0.1196926	Inf	1	-0.5765674	0.8326433
VR / VR Monitor	Type3	0.9989609	0.1174368	Inf	1	-0.0088436	0.9999569
VR / VR Monitor Stereo	Type3	0.8897479	0.1103742	Inf	1	-0.9416859	0.6138546
VR Monitor / VR Monitor Stereo	Type3	0.8906734	0.1164260	Inf	1	-0.8857123	0.6493393

Compare Answer RT

Extract wide dataframe

Do all of the previous, but for the answer reaction times for the selected question types.

We again create a dataframe in needed format of only the answer RT data, filling in na/missing values in the data.

```
#compare answer times

trial_type_means <- all_data_no_outliers %>%
  group_by(ParticipantId, Condition, TrialType) %>%
  summarize(mean_answerRT = mean(AnswerRT), n = n())
```

```
## `summarise()` has grouped output by 'ParticipantId', 'Condition'. You can override using
## the `.groups` argument.
```

```
# first, make .csv files in wide format to double check in statview
answer_rt <- trial_type_means %>%
  select(ParticipantId, Condition, TrialType, mean_answerRT) %>%
  pivot_wider(names_from = TrialType, values_from = mean_answerRT)

# fill in na values with mean for type1, type2 and type3 RT
# the following is supposed to work, but we get NaN from calculation of mean here, not sure why...
#read_rt_type_wider_clean <- read_rt_type_wider %>% mutate(across(Type1, ~replace_na(., mean(., na.rm=T
answer_rt <- answer_rt %>% replace_na(list(Type1 = mean(answer_rt$Type1, na.rm=TRUE)))
answer_rt <- answer_rt %>% replace_na(list(Type2 = mean(answer_rt$Type2, na.rm=TRUE)))
answer_rt <- answer_rt %>% replace_na(list(Type3 = mean(answer_rt$Type3, na.rm=TRUE)))


answer_rt_summary <- answer_rt %>%
  group_by(Condition) %>%
  summarize(meanType1 = mean(Type1, na.rm = T),
            meanType2 = mean(Type2, na.rm = T),
            meanType3 = mean(Type3, na.rm=T))
kable(answer_rt_summary)
```

Condition	meanType1	meanType2	meanType3
VR	27634.04	39506.10	59359.30
VR Monitor	31197.76	40741.96	62175.44
VR Monitor Stereo	29336.88	35546.24	52789.19

```
rm(answer_rt_summary)

#rm(trial_type_means)
```

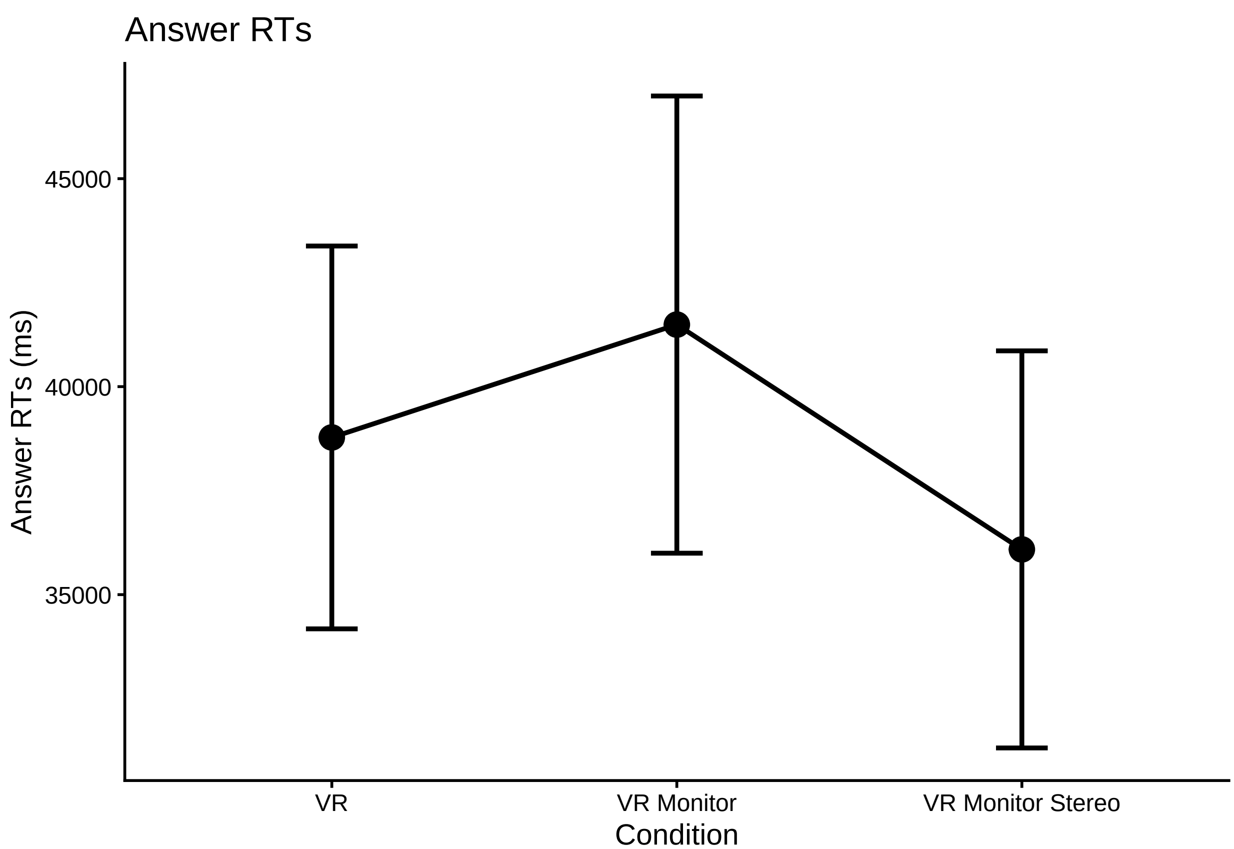
Basic plots of main effect and wider format

Plot of answer RT condition main effect, as well as wider format plot by the trial types.

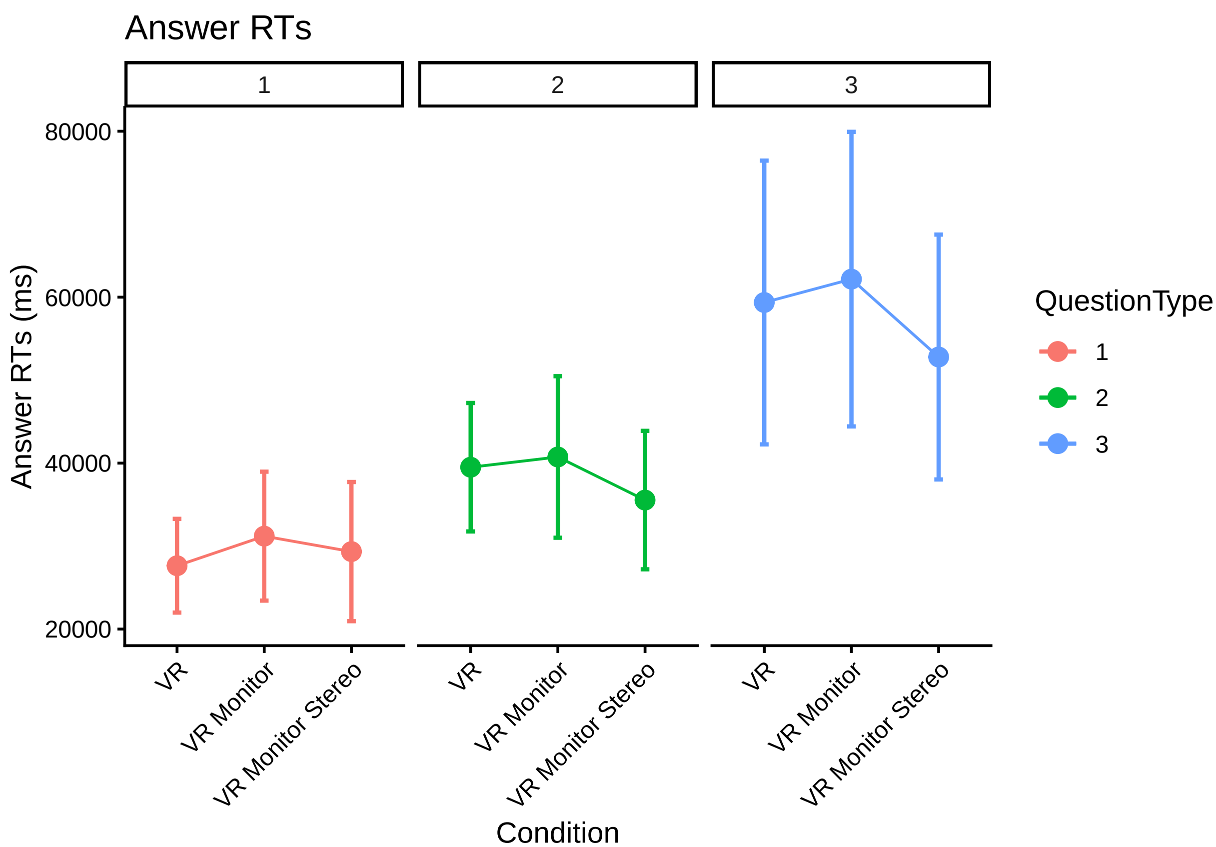
```
#just condition main effect
answerplot1 <- all_data_no_outliers %>%
  group_by(ParticipantId, Condition) %>%
  summarize(overallmean = mean(AnswerRT)) %>%
  group_by(Condition) %>%
  summarize(overall_condition_mean = mean(overallmean),
            se = std.error(overallmean),
            n = n(),
            CI = qt(0.975,df=n-1)*se)
```

```
## `summarise()` has grouped output by 'ParticipantId'. You can override using the `.groups`
## argument.
```

```
ggplot(answerplot1, aes(Condition,
                        overall_condition_mean,
                        group = 1,
                        ymin = overall_condition_mean - CI,
                        ymax = overall_condition_mean + CI)) +
  theme_classic() +
  geom_point(size = 4) +
  geom_errorbar(width = .15, linewidth = 0.85) +
  geom_line(linewidth = 0.85) +
  labs(y = "Answer RTs (ms)", title = "Answer RTs")
```



```r
rm(answerplot1)

#make a little plot using wide format
superbPlot(answer_rt,
    BSFactors    = "Condition",
    WSFactors    = "QuestionType(3)",
    variables    = c("Type1", "Type2", "Type3"),
    statistic    = "meanNArm",
    errorbar     = "CI",
    gamma        = 0.95,
    adjustments = list(
        purpose       = "difference"
    ),
    plotLayout = "line",
    factorOrder = c("Condition", "QuestionType")
) +
theme_classic() +
  theme(axis.text.x = element_text(angle = 45, vjust = 1, hjust=1))+
facet_wrap(vars(QuestionType))+
labs(y = "Answer RTs (ms)", title = "Answer RTs")
```

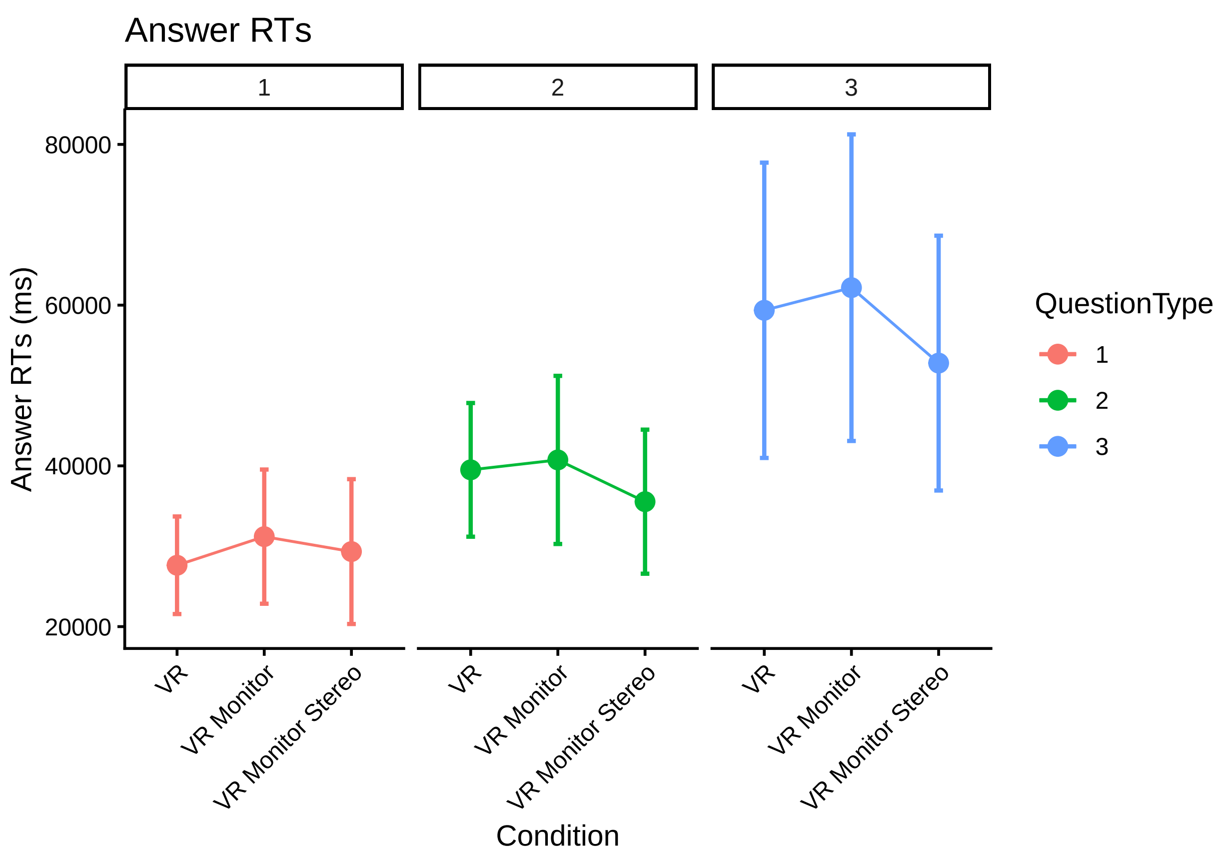


Correct wider plot heterogeneous variance

The message from the superbPlot may indicate that some of the groups' variances are heterogeneous. We can try using the `tryon` purpose option as suggested, read Unequal variances, Welch test, Tryon adjustment, and superb.

```r
# make the wide plot, use tryon for purpose as suggested because of group's heterogeneous variances
superbPlot(answer_rt,
    BSFactors    = "Condition",
    WSFactors    = "QuestionType(3)",
    variables    = c("Type1", "Type2", "Type3"),
    statistic    = "meanNArm",
    errorbar     = "CI",
    gamma        = 0.95,
    adjustments = list(
        purpose       = "tryon"
    ),
    plotLayout = "line",
    factorOrder = c("Condition", "QuestionType")
) +
theme_classic() +
  theme(axis.text.x = element_text(angle = 45, vjust = 1, hjust=1))+
facet_wrap(vars(QuestionType))+
labs(y = "Answer RTs (ms)", title = "Answer RTs")
```

```
## superb::FYI: The tryon adjustments per measures are  Measure 1: 1.5195, Measure 2: 1.5195, Measure 3
```



Perform anova and post-hoc tests

Finally perform anova on the answer RT data and various posthoc tests.

```r
# need back into long format for following, but we will use out dataframe
# will filled in na/missing and make wide again here
```

```r
answer_rt_long <- answer_rt %>%
     pivot_longer(Type1:Type3, names_to="TrialType", values_to = "mean_answerRT")

answer_rt_anova <- aov_ez(id = "ParticipantId",
                          dv = "mean_answerRT",
                          data = answer_rt_long,
                          within = "TrialType",
                          between = "Condition",
                          anova_table = list(es = "pes") #might want to double-check these
)
```

```
## Converting to factor: Condition

## Contrasts set to contr.sum for the following variables: Condition
```

```r
kable(nice(answer_rt_anova))
```

Effect	df	MSE	F	pes	p.value
Condition	2, 119	972114036.18	0.83	.014	.439
TrialType	1.47, 175.41	749109866.00	46.16 ***	.279	<.001
Condition:TrialType	2.95, 175.41	749109866.00	0.38	.006	.767

```r
#posthoc test for trial type
pairs(emmeans(answer_rt_anova, "TrialType"), adjust = "Tukey")
```

```
##  contrast      estimate   SE  df t.ratio p.value
##  Type1 - Type2    -9209 1940 119  -4.737  <.0001
##  Type1 - Type3   -28718 3540 119  -8.104  <.0001
##  Type2 - Type3   -19510 3410 119  -5.724  <.0001
##
## Results are averaged over the levels of: Condition
## P value adjustment: tukey method for comparing a family of 3 estimates
```

```r
#posthoc test for condition, still using Tukey
pairs(emmeans(answer_rt_anova, "Condition"), adjust = "Tukey")
```

```
##  contrast                        estimate   SE  df t.ratio p.value
##  VR - VR Monitor                    -2539 3850 119  -0.660  0.7870
##  VR - VR Monitor Stereo              2942 4040 119   0.729  0.7469
##  VR Monitor - VR Monitor Stereo      5481 4260 119   1.287  0.4051
##
## Results are averaged over the levels of: TrialType
## P value adjustment: tukey method for comparing a family of 3 estimates
```

```r
#posthoc test for condition, others seem to be scheffe, sidak, bonferroni, dunnettx, mvt, none
pairs(emmeans(answer_rt_anova, "Condition"), adjust = "scheffe")
```

```
##  contrast                        estimate   SE  df t.ratio p.value
##  VR - VR Monitor                    -2539 3850 119  -0.660  0.8046
##  VR - VR Monitor Stereo              2942 4040 119   0.729  0.7672
##  VR Monitor - VR Monitor Stereo      5481 4260 119   1.287  0.4392
##
## Results are averaged over the levels of: TrialType
## P value adjustment: scheffe method with rank 2
```

```r
rm(answer_rt_long)
rm(answer_rt_anova)
```

Alternative Analysis using glmm TMB package

The following analysis is for the heterogenous group condition analysis. See .

The following determines if a more complex model is justified.

```r
model_glmm_hom = glmmTMB(mean_answerRT ~ Condition * TrialType + (1 | ParticipantId),
                         data=trial_type_means, family = Gamma(link="log"),
                         dispformula = ~ 1, REML = FALSE)
model_glmm_hetero = glmmTMB(mean_answerRT ~ Condition * TrialType + (1 | ParticipantId),
                         data=trial_type_means, family = Gamma(link="log"),
                         dispformula = ~ Condition, REML = FALSE)


anova(model_glmm_hom, model_glmm_hetero)
```

```
## Data: trial_type_means
## Models:
## model_glmm_hom: mean_answerRT ~ Condition * TrialType + (1 | ParticipantId), zi=~0, disp=~1
## model_glmm_hetero: mean_answerRT ~ Condition * TrialType + (1 | ParticipantId), zi=~0, disp=~Conditi
##                   Df    AIC    BIC  logLik deviance  Chisq Chi Df Pr(>Chisq)
## model_glmm_hom    11 8183.7 8226.6 -4080.9   8161.7
## model_glmm_hetero 13 8187.6 8238.2 -4080.8   8161.6 0.1269      2     0.9385
```

Try overall effect reporting

```r
fixed_effects_anova <- Anova(model_glmm_hom, type="III")
kable(fixed_effects_anova)
```

	Chisq	Df	Pr(>Chisq)
(Intercept)	1.650701e+04	1	0.000000
Condition	9.737463e-01	2	0.614545
TrialType	5.497504e+01	2	0.000000
Condition:TrialType	1.093754e+00	4	0.895262

EMM on Condition and Interaction

If the glmm analysis is not significant, just use the homogeneous model. Otherwise report on the more complex heterogeneous model?

```r
emm_main_condition <- emmeans(model_glmm_hom,
                        specs = ~ Condition,
                        type = "response")
```

```
## NOTE: Results may be misleading due to involvement in interactions
```

```r
condition_results <- pairs(emm_main_condition)


kable(condition_results)
```

contrast	ratio	SE	df	null	z.ratio	p.value
VR / VR Monitor	0.9297006	0.0808360	Inf	1	-0.8383431	0.6791495
VR / VR Monitor Stereo	1.0516902	0.0962689	Inf	1	0.5505792	0.8461812
VR Monitor / VR Monitor Stereo	1.1312138	0.1091549	Inf	1	1.2777132	0.4077786

```
emm_interaction <- emmeans(model_glmm_hom,
                           specs = ~ Condition * TrialType,
                           type = "response")

simple_effects_results <- pairs(emm_interaction, by = "TrialType")


kable(simple_effects_results)
```

contrast	TrialType	ratio	SE	df	null	z.ratio	p.value
VR / VR Monitor	Type1	0.8894706	0.1067392	Inf	1	-0.9760491	0.5920344
VR / VR Monitor Stereo	Type1	0.9660220	0.1218370	Inf	1	-0.2740886	0.9594340
VR Monitor / VR Monitor Stereo	Type1	1.0860640	0.1445458	Inf	1	0.6203267	0.8089789
VR / VR Monitor	Type2	0.9734983	0.1165213	Inf	1	-0.2243998	0.9726224
VR / VR Monitor Stereo	Type2	1.0970811	0.1378832	Inf	1	0.7372036	0.7413456
VR Monitor / VR Monitor Stereo	Type2	1.1269471	0.1493863	Inf	1	0.9015824	0.6392970
VR / VR Monitor	Type3	0.9280313	0.1130761	Inf	1	-0.6129902	0.8130181
VR / VR Monitor Stereo	Type3	1.0975838	0.1416871	Inf	1	0.7212895	0.7508719
VR Monitor / VR Monitor Stereo	Type3	1.1827014	0.1607484	Inf	1	1.2345912	0.4327095

Compare Accuracy

Extract wide dataframe

Do all of the previous again but on the Accuracy results.

We again create a dataframe in needed format of only the accuracy data.
A bit different from before, only need to create an avverage of the two correct features we have. But then we do a similar transformation into wide format and replace na/missing values.

```
all_data_no_outliers <- all_data_no_outliers %>%
  rowwise() %>%
  mutate(Correct_Avg = mean(c(Correct_1, Correct_2)))

trial_type_mean_accuracy <- all_data_no_outliers %>%
  group_by(ParticipantId, Condition, TrialType) %>%
  summarize(mean_acc = mean(Correct_Avg),
            n = n())
```

```
## `summarise()` has grouped output by 'ParticipantId', 'Condition'. You can override using
## the `.groups` argument.
```

```
accuracy <- trial_type_mean_accuracy %>%
  select(ParticipantId, Condition, TrialType, mean_acc) %>%
  pivot_wider(names_from = TrialType, values_from = mean_acc)
```

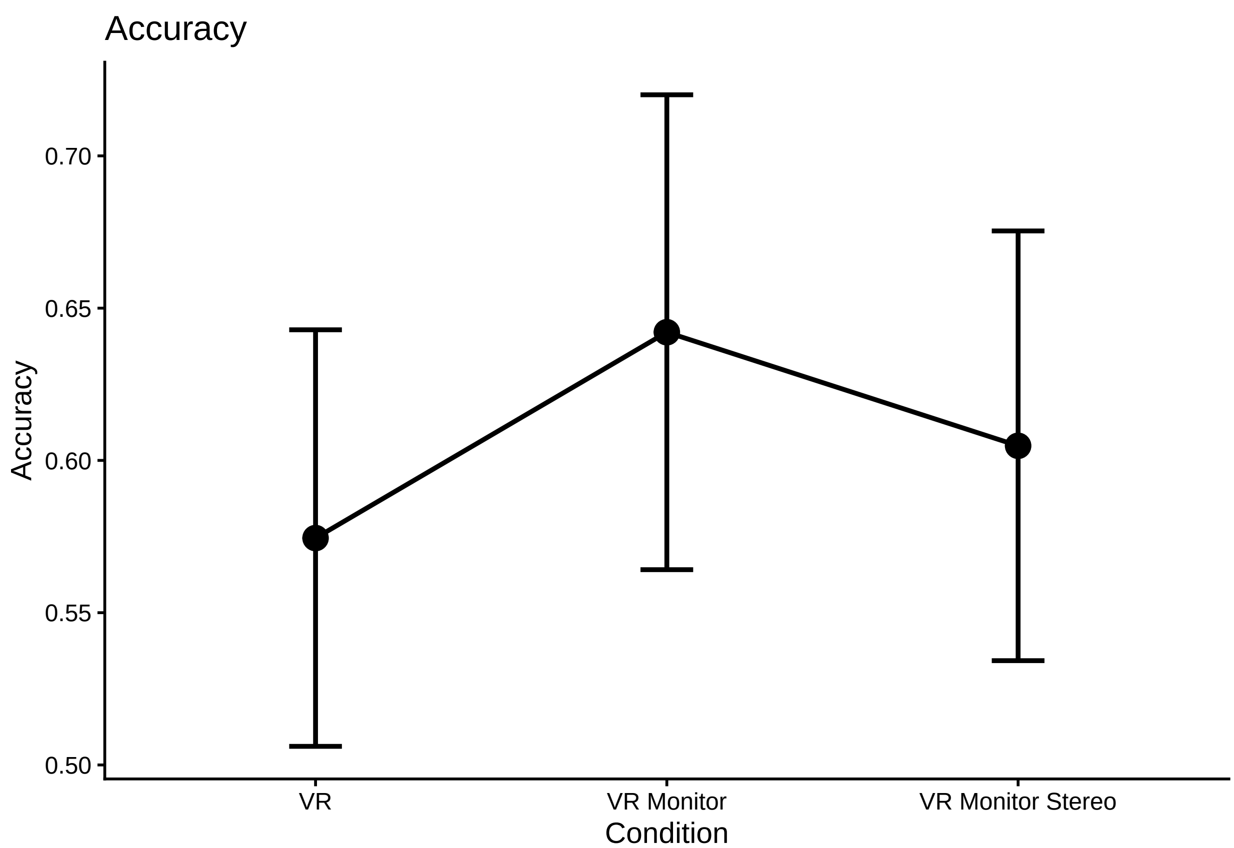
```r
# replace na with mean
accuracy <- accuracy %>% replace_na(list(Type1 = mean(accuracy$Type1, na.rm=TRUE)))
accuracy <- accuracy %>% replace_na(list(Type2 = mean(accuracy$Type2, na.rm=TRUE)))
accuracy <- accuracy %>% replace_na(list(Type3 = mean(accuracy$Type3, na.rm=TRUE)))

rm(trial_type_mean_accuracy)
```

Basic plots of main effect and wider format

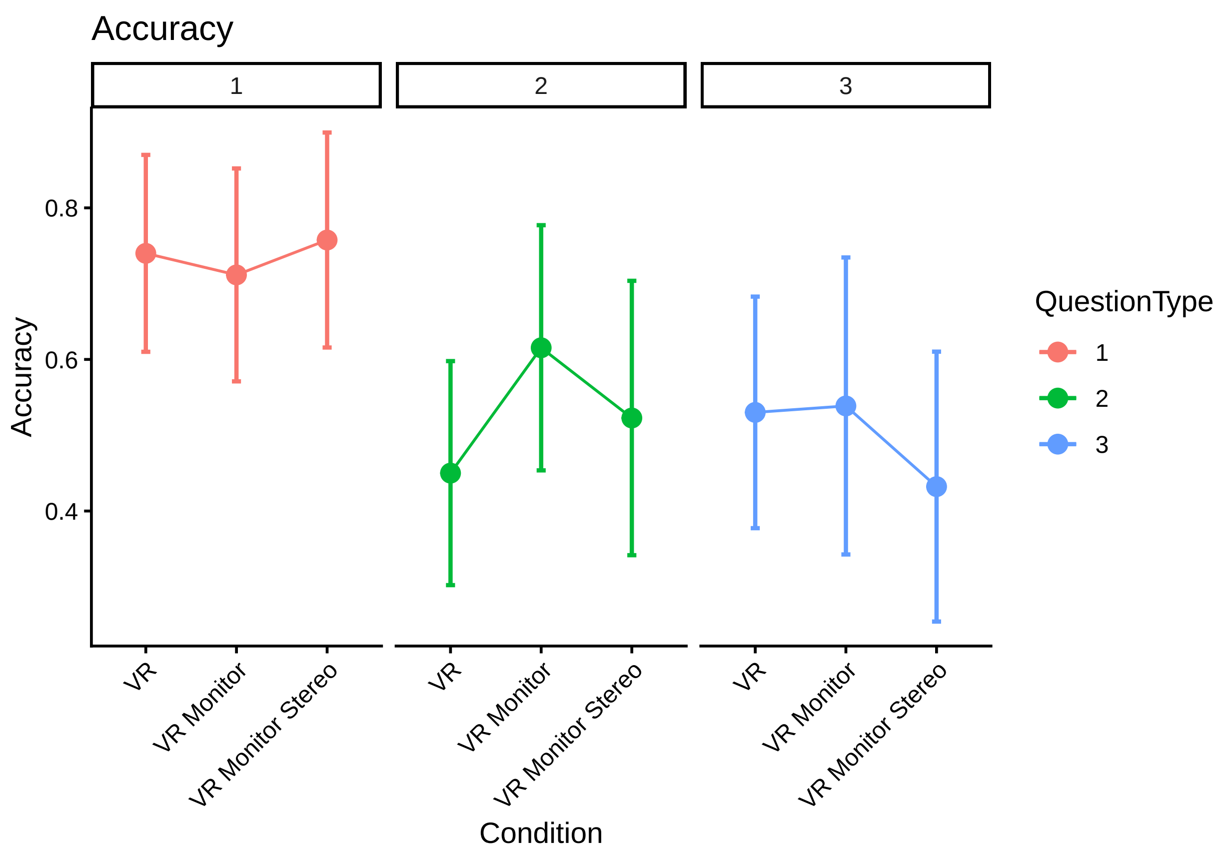
```r
#just condition main effect
accplot1 <- all_data_no_outliers %>%
   group_by(ParticipantId, Condition) %>%
   summarize(overallmean = mean(Correct_Avg)) %>%
   group_by(Condition) %>%
   summarize(overall_condition_mean = mean(overallmean),
             se = std.error(overallmean),
             n = n(),
             CI = qt(0.975,df=n-1)*se)
```

```
## `summarise()` has grouped output by 'ParticipantId'. You can override using the `.groups`
## argument.
```

```r
ggplot(accplot1, aes(Condition,
                       overall_condition_mean,
                       group = 1,
                       ymin = overall_condition_mean - CI,
                       ymax = overall_condition_mean + CI)) +
   theme_classic() +
   geom_point(size = 4) +
   geom_errorbar(width = .15, linewidth = 0.85) +
   geom_line(linewidth = 0.85) +
   labs(y = "Accuracy", title = "Accuracy")
```




```r
rm(accplot1)

#make the little plot
superbPlot(accuracy,
    BSFactors    = "Condition",
    WSFactors    = "QuestionType(3)",
    variables    = c("Type1", "Type2", "Type3"),
    statistic    = "meanNArm",
    errorbar     = "CI",
    gamma        = 0.95,
    adjustments = list(
        purpose        = "difference"
    ),
    plotLayout = "line",
    factorOrder = c("Condition", "QuestionType")
) +
theme_classic() +
theme(axis.text.x = element_text(angle = 45, vjust = 1, hjust=1))+
facet_wrap(vars(QuestionType))+
labs(y = "Accuracy", title = "Accuracy")
```

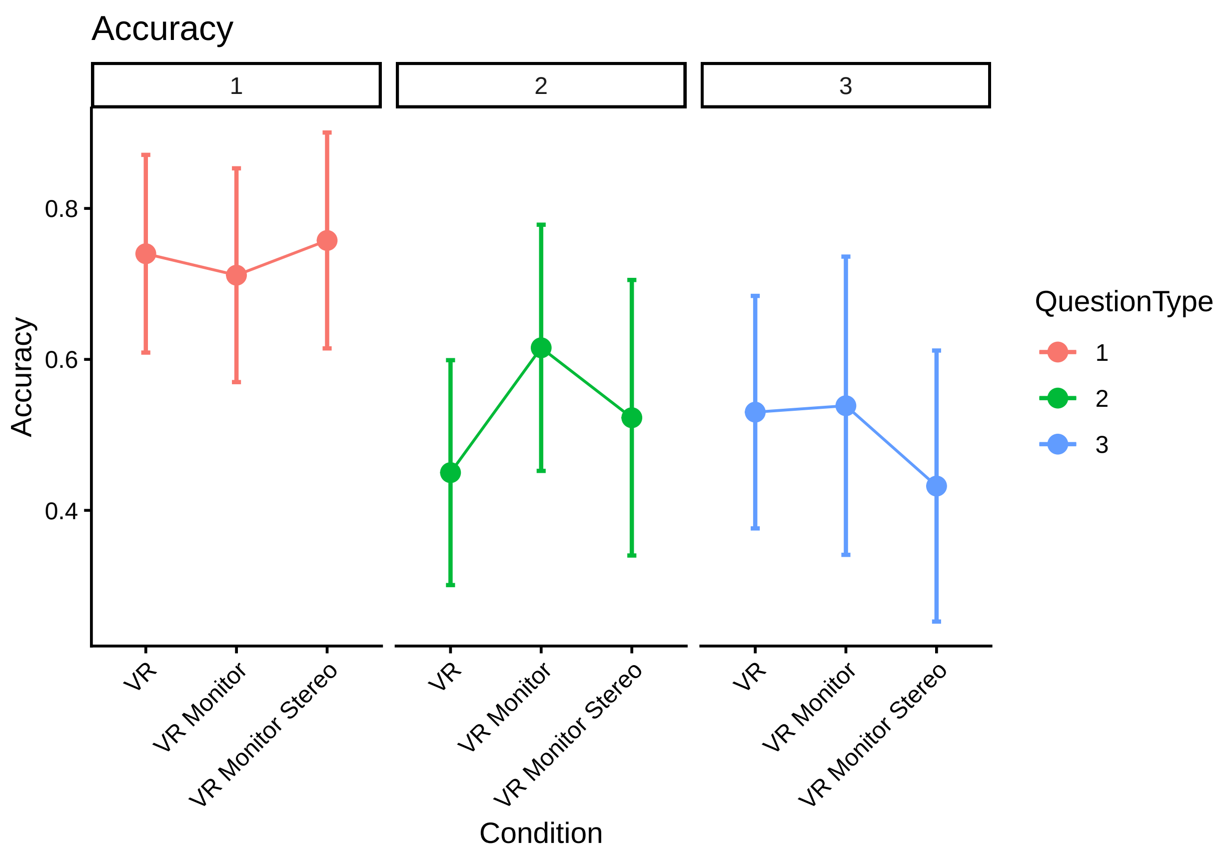


Correct wider plot heterogeneous variance

The message from the superbPlot may indicate that some of the groups' variances are heterogeneous. We can try using the `tryon` purpose option as suggested, read Unequal variances, Welch test, Tryon adjustment, and superb.

```r
# make the wide plot, use tryon for purpose as suggested because of group's heterogeneous variances
superbPlot(accuracy,
    BSFactors    = "Condition",
    WSFactors    = "QuestionType(3)",
    variables    = c("Type1", "Type2", "Type3"),
    statistic    = "meanNArm",
    errorbar     = "CI",
    gamma        = 0.95,
    adjustments = list(
        purpose       = "tryon"
    ),
    plotLayout = "line",
    factorOrder = c("Condition", "QuestionType")
) +
theme_classic() +
  theme(axis.text.x = element_text(angle = 45, vjust = 1, hjust=1))+
facet_wrap(vars(QuestionType))+
labs(y = "Accuracy", title = "Accuracy")
```

```
## superb::FYI: The tryon adjustments per measures are  Measure 1: 1.4258, Measure 2: 1.4258, Measure 3
```



Perform anova and post-hoc tests

Finally perform anova on the accuracy data and various posthoc tests.

```
# need back into long format for following, but we will use out dataframe
# will filled in na/missing and make wide again here
accuracy_long <- accuracy %>%
     pivot_longer(Type1:Type3, names_to="TrialType", values_to = "mean_acc")

accuracy_anova <- aov_ez(id = "ParticipantId",
                          dv = "mean_acc",
                          data = accuracy_long,
                          within = "TrialType",
                          between = "Condition",
                          anova_table = list(es = "pes") #might want to double-check these
)
```

```
## Converting to factor: Condition
```

```
## Contrasts set to contr.sum for the following variables: Condition
```

```
kable(nice(accuracy_anova))
```

Effect	df	MSE	F	pes	p.value
Condition	2, 119	0.17	0.58	.010	.562
TrialType	1.98, 235.91	0.10	18.87 ***	.137	<.001
Condition:TrialType	3.96, 235.91	0.10	1.67	.027	.159

```r
#posthoc test for trial type
pairs(emmeans(accuracy_anova, "TrialType"), adjust = "Tukey")
```

```
##  contrast      estimate     SE  df t.ratio p.value
##  Type1 - Type2   0.2070 0.0432 119   4.788  <.0001
##  Type1 - Type3   0.2361 0.0399 119   5.910  <.0001
##  Type2 - Type3   0.0291 0.0425 119   0.684  0.7732
##
## Results are averaged over the levels of: Condition
## P value adjustment: tukey method for comparing a family of 3 estimates
```

```r
#posthoc test for condition, still using Tukey
pairs(emmeans(accuracy_anova, "Condition"), adjust = "Tukey")
```

```
##  contrast                        estimate     SE  df t.ratio p.value
##  VR - VR Monitor                 -0.04847 0.0506 119  -0.957  0.6054
##  VR - VR Monitor Stereo           0.00254 0.0532 119   0.048  0.9987
##  VR Monitor - VR Monitor Stereo   0.05101 0.0561 119   0.910  0.6352
##
## Results are averaged over the levels of: TrialType
## P value adjustment: tukey method for comparing a family of 3 estimates
```

```r
#posthoc test for condition, others seem to be scheffe, sidak, bonferroni, dunnettx, mvt, none
pairs(emmeans(accuracy_anova, "Condition"), adjust = "scheffe")
```

```
##  contrast                        estimate     SE  df t.ratio p.value
##  VR - VR Monitor                 -0.04847 0.0506 119  -0.957  0.6337
##  VR - VR Monitor Stereo           0.00254 0.0532 119   0.048  0.9989
##  VR Monitor - VR Monitor Stereo   0.05101 0.0561 119   0.910  0.6621
##
## Results are averaged over the levels of: TrialType
## P value adjustment: scheffe method with rank 2
```

```r
rm(accuracy_long)
rm(accuracy_anova)
```